HW3 - STAT 701

Wells Guan

Date: October 29, 2026

Preface: for all multitaper/DPSS related questions, you are welcome to use the R package `multitaper` (or python/Julia equivalents). You are also welcome to use chatbots or other external tools for hard Fourier integrals. It is not my intention for you to spend hours grinding out $\int_{-1/2}^{1/2} e^{i\omega h} e^{-\theta|\omega|}\, \mathrm{d}\omega$.

Problem 1

Consider a sample path of length n from a zero-mean, stationary time series $\{X_t\}_{t=1}^n$. Define its weighted Discrete Fourier Transform (DFT) at the Fourier frequencies $f_k = k/n$ for $k \in \{-[(n-1)/2], \ldots, [n/2]\}$, where $[\cdot]$ denotes the integer part of a number, by

$$\widetilde{X}(f_k) = \sum_{t=1}^{n} w_t X_t e^{-i2\pi f_k t}$$

for a sequence of real-valued data taper weights $\{w_t\}_{t=1}^n$.

1. Let $w_t = n^{-1/2}$ for all t, so that $|\widetilde{X}(f_k)|^2 = I(f_k)$ is the standard un-tapered periodogram estimator. Pick $n = 512$ and a known autocovariance function $\gamma(h)$ for your process (e.g., an ARMA(p,q)). Using simulation:
   (a) Draw $m \gg n$ independent sample paths (e.g., $m \approx 10n$).
   (b) Compute the vector of DFT coefficients $\widetilde{\mathbf{X}}^{(i)} = [\widetilde{X}^{(i)}(f_k)]$ for Fourier frequency f_k defined above for each realization $i = 1, \ldots, m$.
   (c) Construct the empirical covariance matrix $\widehat{\Sigma} \in \mathbb{C}^{n\times n} = \sum_{i=1}^m \widetilde{\mathbf{X}}^{(i)} \widetilde{\mathbf{X}}^{(i)H}$ of these DFT coefficients.

   Inspect the matrix $\widehat{\Sigma}$ and comment on the resulting structure: how correlated are the estimates across different Fourier frequencies f_k and $f_{k'}$? If you were interested to make a diagonal approximation of $\widehat{\Sigma}$, what information would you be throwing away?

   Sol.

   (a) The code for drawing (ARMA) sample paths:

```
samples = np.stack([arma_generate_sample(
    ar=[1, -0.6],
    ma=[1, 0.3],
    nsample=N,
    scale=1,
    burnin=500,
    distrvs=rng.standard_normal,
    ) for i in range(M)])
```

(b) The code for computing DFT vectors for a single sample path:

```
def simulated_paths(times_series, alpha = 10, rng=None):
    n = times_series.shape[0]
    weights = np.ones(n)/(n ** 0.5)
    frequencies = np.arange(-np.floor((n-1)/2),np.floor(n/2)+1)/n
    dft = np.stack([weighted_dft(times_series, weights, frequency) for frequency in
    frequencies])
    return dft

def weighted_dft(paths, weights, frequency):
    time = np.arange(1,paths.shape[0]+1)
    exp_term = np.exp( - 1j * 2 * np.pi * frequency * time)
    return np.sum(weights * paths * exp_term)
```

(c) The code of constructing:

```
def empirical_dft_covariance(coefficients):
    dft_covaiance = np.outer(coefficients[0], coefficients[0].conj())
    for coefficient in coefficients[1:]:
        dft_covaiance += np.outer(coefficient, coefficient.conj())
    return dft_covaiance
```

and the structure of the matrix is shown the following:

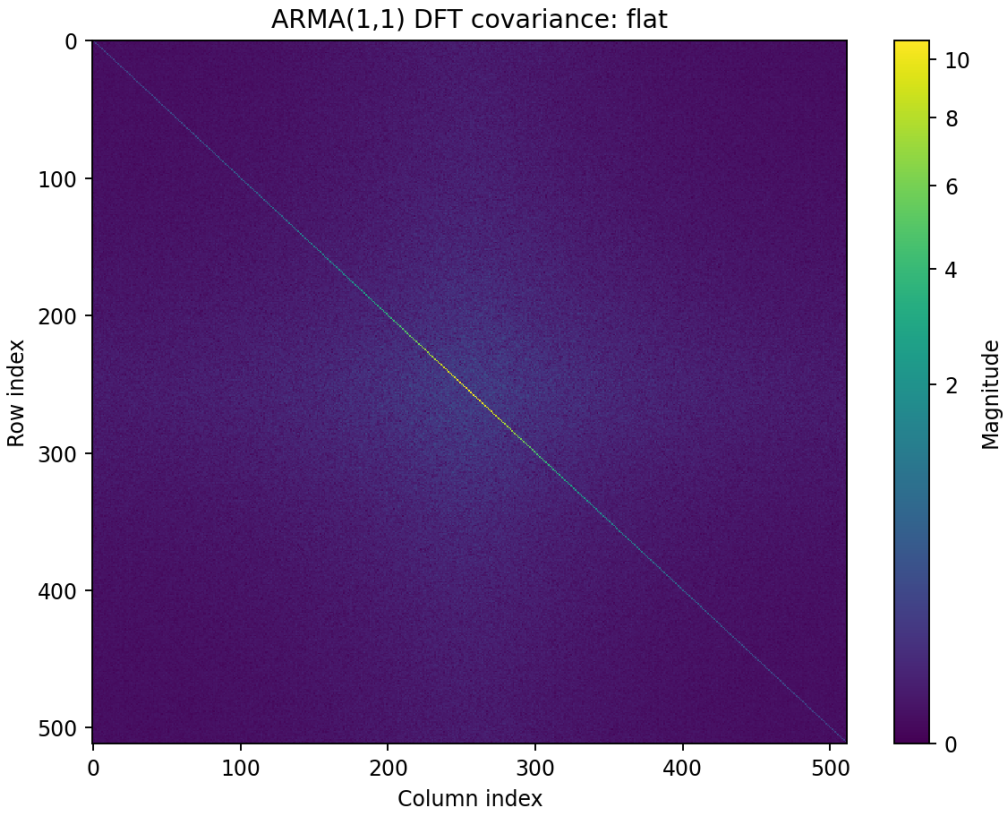


Figure 1: Empirical covariance matrix of the DFT coefficients.

I used the zero-mean Gaussian ARMA(1, 1) model

$$X_t = 0.6X_{t-1} + Z_t + 0.3Z_{t-1}, \qquad Z_t \sim N(0, 1).$$

Its known autocovariance is $\gamma(0) = 2.265625$ and $\gamma(h) = 1.659375(0.6)^{|h|-1}$ for $|h| \geq 1$. I generated $m = 5120$ independent paths of length $n = 512$, discarding 500 initial observations. The code above returns the sum in the question; for the figures I divide by m to estimate the population covariance. This rescaling does not change its structure.

The matrix is approximately diagonal, with the largest diagonal variances near frequency zero. Large covariance magnitude does not by itself mean large correlation. To separate these effects, I also use

$$\widehat{R}_{k\ell} = \frac{\widehat{\Sigma}_{k\ell}}{\sqrt{\widehat{\Sigma}_{kk}\widehat{\Sigma}_{\ell\ell}}}.$$

For this model the mean magnitude of the theoretical adjacent-frequency correlations is 0.00606, compared with 0.01469 in the simulation. Some empirical off-diagonal values are Monte Carlo error: they need not vanish in a finite sample of 5120 paths. A diagonal approximation retains frequency-specific variances and discards all cross-frequency covariances, including finite-window boundary effects. For the exact covariance its relative Frobenius approximation error is 0.04367; thus the error is small in this example, rather than evidence of strong coupling to zero frequency. Because the data are real, opposite-frequency DFT coefficients are complex conjugates. A diagonal Hermitian covariance alone does not imply independence of the entire two-sided complex DFT vector.

2. Repeat this exercise using a normalized Slepian/discrete prolate spheroidal sequence (DPSS) taper sequence $w_t = h_t^{(0)}$ with time-bandwidth parameter $NW = 10$. What structural differences do you observe in the empirical covariance matrix compared to the un-tapered case?

Sol.

I repeat the experiment with the same ARMA coefficients and the same 5120 sample paths, changing only the taper. The weights are obtained by

```
weights = dpss(N, NW=10, Kmax=1, norm=2)[0]
```

so that $\sum_t w_t^2 = 1$. The wider band around the diagonal represents strong correlations between nearby Fourier coefficients.

For a white-noise input with variance σ^2, the tapered covariance is

$$\mathrm{Cov}(\widetilde{X}(f_k), \widetilde{X}(f_\ell)) = \sigma^2 \sum_{t=1}^{n} w_t^2 e^{-2\pi i(f_k - f_\ell)t}.$$

Flat weights make this zero at distinct Fourier frequencies, but DPSS weights do not. The taper improves spectral concentration while sacrificing the orthogonality of adjacent weighted Fourier coefficients. In this ARMA experiment, the mean theoretical adjacent correlation magnitude is 0.92436 (empirical 0.92435), and the relative Frobenius error of a diagonal approximation is 0.88070. The normalized correlation plot makes the band visible away from the high-power central region as well.

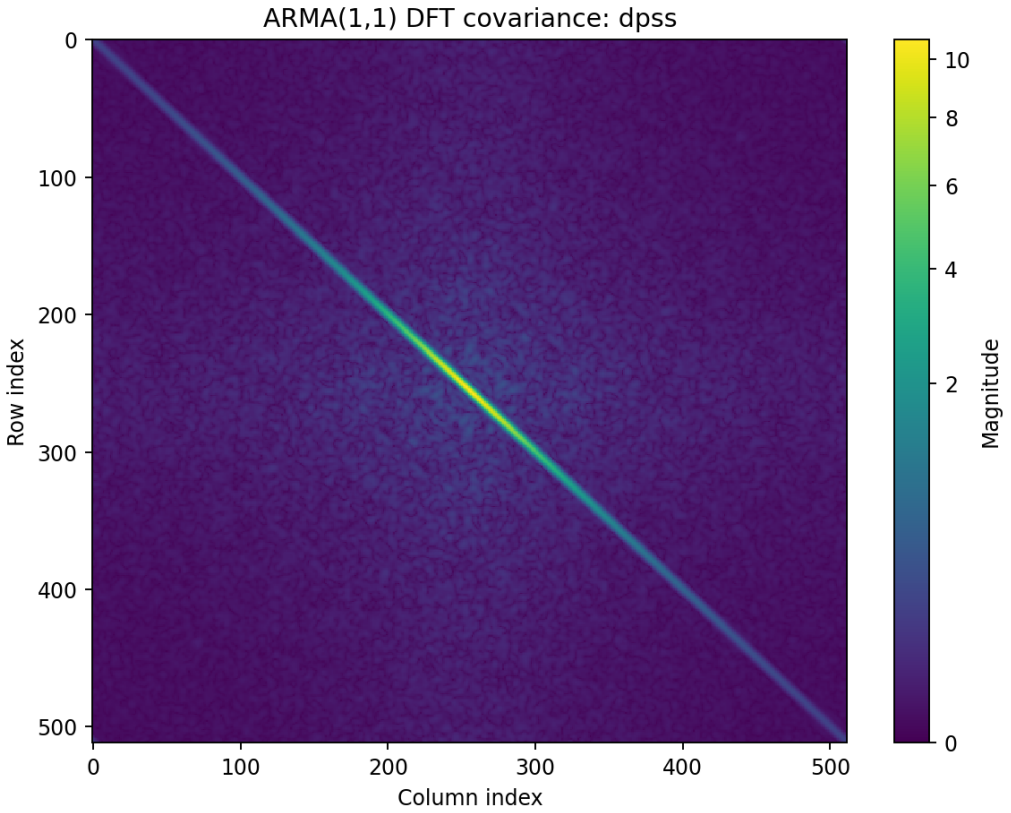


Figure 2: Empirical covariance matrix of the DFT coefficients with DPSS weights.

3*. Analytically derive the cross-covariance $\mathrm{Cov}(\widetilde{X}(f_k), \widetilde{X}(f_{k'}))$ for the flat weight sequence $w_t = n^{-1/2}$ in terms of the process spectral density $S(f)$ or autocovariance $\gamma(h)$. Simplify as much as possible using Dirichlet kernel properties. Based on your result, at what rate do you think these covariances go to zero?

Sol.

I use the complex covariance convention $\mathrm{Cov}(U, V) = \mathbb{E}[U\overline{V}]$ for centered variables, and frequencies in cycles per observation, so $\gamma(h) = \int_{-1/2}^{1/2} S(f) e^{2\pi i f h}\, df$. For $D_n(u) = \sum_{t=1}^{n} e^{-2\pi i u t}$,

$$D_n(u) = e^{-\pi i (n+1) u} \frac{\sin(\pi n u)}{\sin(\pi u)},$$

with its continuous extension at integers. Expanding the covariance gives

$$\begin{aligned} C_{k\ell} &= \frac{1}{n} \sum_{t,s=1}^{n} \gamma(t-s) e^{-2\pi i (f_k t - f_\ell s)} \\ &= \frac{1}{n} \int_{-1/2}^{1/2} S(f) D_n(f_k - f) \overline{D_n(f_\ell - f)}\, df \\ &= \frac{1}{n} \sum_{|h|<n} \gamma(h) e^{-2\pi i f_k h} \sum_{s=\max(1,1-h)}^{\min(n,n-h)} e^{-2\pi i (f_k - f_\ell) s}. \end{aligned}$$

For $k = \ell$, the inner sum is $n - |h|$, giving the Fejer-smoothed spectrum

$$C_{kk} = \sum_{|h|<n} \left(1 - \frac{|h|}{n}\right) \gamma(h) e^{-2\pi i f_k h}.$$

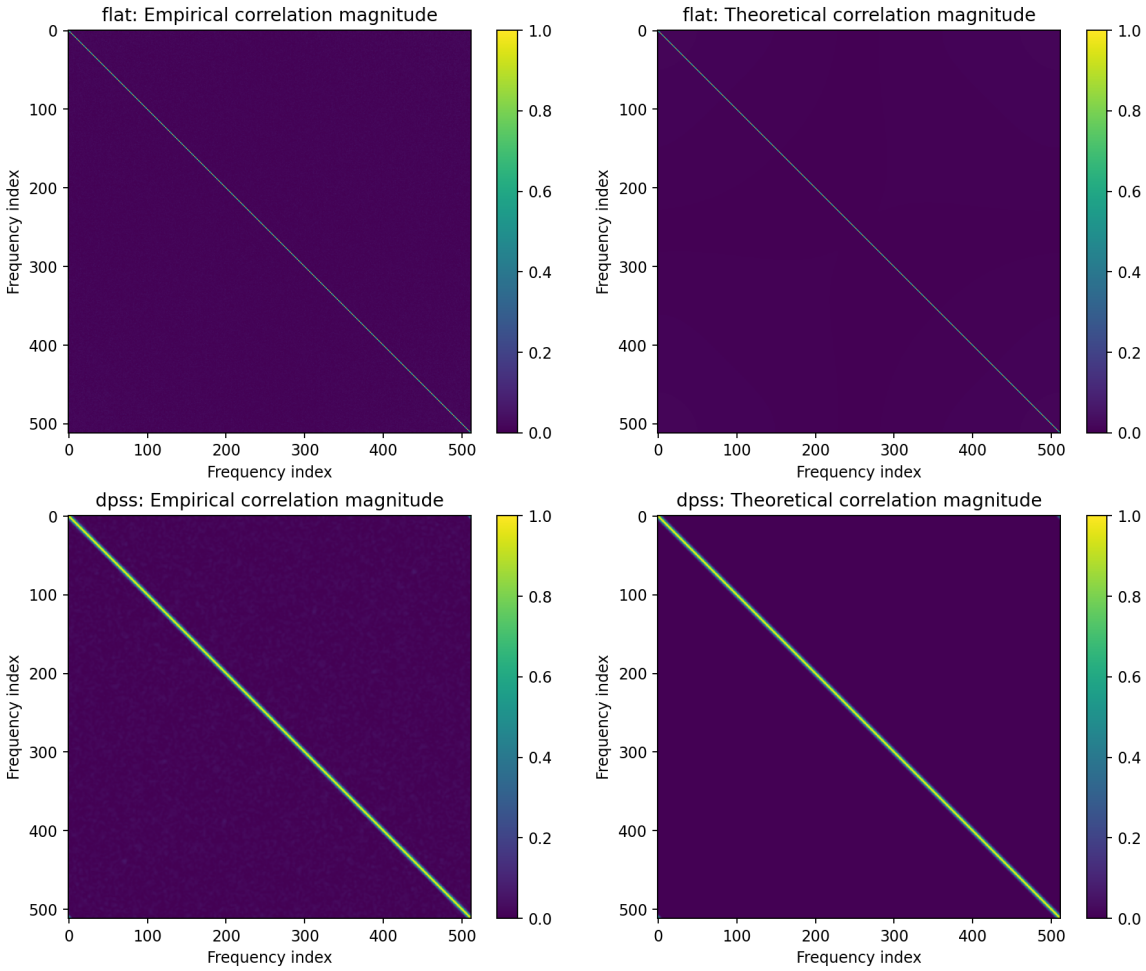


Figure 3: Empirical and exact correlation magnitudes for the same ARMA process. All panels use the same linear color scale from zero to one.

For distinct Fourier frequencies modulo one, $\sum_{s=1}^{n} e^{-2\pi i(f_k - f_\ell)s} = 0$. The shortened sum omits only $|h|$ terms, so

$$|C_{k\ell}| \le \frac{1}{n} \sum_{|h|<n} |h|\,|\gamma(h)|.$$

Consequently, if $\sum_h |h|\,|\gamma(h)| < \infty$, as for our causal ARMA model, the off-diagonal covariances are uniformly $O(n^{-1})$. Absolute summability of γ alone implies $o(1)$ by dominated convergence, but need not give this rate. Stationarity alone does not imply vanishing cross-frequency covariance or a universal rate. For white noise the off-diagonal covariances are exactly zero.

Problem 2

Let's explore the local versus broadband bias tradeoffs in spectral density estimation.

1. Write code that uses circulant embedding to *exactly* simulate a length-n sample path from a stationary time series with arbitrary covariance function $\gamma(\cdot)$. Please design its interface to accept an ACF sequence $\{\gamma(0), \dots, \gamma(n-1)\}$ and return the length-n sequence. You may use a built-in library for the FFT, but please do the other book-keeping yourself.

   Sol.

   The code uses the symmetric circulant first row

   $$c = (\gamma(0), \dots, \gamma(n-1), \gamma(n-2), \dots, \gamma(1)), \qquad L = 2(n-1).$$

   Its eigenvalues are $\lambda = \mathrm{FFT}(c)$. With a real vector $z \sim N(0, I_L)$, I compute

   $$y = \mathrm{IFFT}\big(\sqrt{\lambda}\ \mathrm{FFT}(z)\big).$$

   The FFT and IFFT normalizations cancel to give $\mathrm{Cov}(y) = F^* \,\mathrm{diag}(\lambda) F / L$, the circulant covariance with first row c. Its first n coordinates therefore have exactly the required Toeplitz covariance, without burn-in. Here is the main implementation:

```
row = np.r_[acf, acf[-2:0:-1]]
eigenvalues = np.fft.fft(row).real
white_noise = rng.standard_normal(len(row))
sample = np.fft.ifft(
    np.sqrt(eigenvalues) * np.fft.fft(white_noise)
).real
return sample[:len(acf)]
```

   This construction requires nonnegative circulant eigenvalues. An arbitrary valid Toeplitz covariance does not guarantee that this particular embedding is positive semidefinite. The full code checks this condition, clips only roundoff-sized negative eigenvalues, and otherwise raises an error rather than silently changing the covariance. A longer valid covariance extension or another exact Gaussian method would then be needed. All embeddings used below passed the check. As a numerical validation, 20,000 independent draws with $n = 8$ and $\gamma(h) = 0.7^{|h|}$ gave a maximum covariance-entry error of 0.0251 against the exact Toeplitz matrix.

2. Recall the periodogram and the multitaper estimator described in class. Please design spectral densities/covariance functions to achieve the following failure modes, and demonstrate them by simulating data and computing the estimators:

   (a) The multitaper estimator misses two distinct peaks and only reports one, but the periodogram gets them both.

   Sol.

   I use $n = 4096$ and

   $$\gamma(h) = 0.994^{|h|}\{\cos(2\pi\, 0.160h) + \cos(2\pi\, 0.164h)\} + 0.05\, 1_{\{h=0\}}.$$

   For each component, the corresponding spectrum is the sum of two shifted Poisson kernels:

   $$S_j(f) = \frac{1}{2} \sum_{\epsilon\in\{-1,1\}} \frac{1-r^2}{1+r^2-2r\cos(2\pi(f-\epsilon f_j))}, \qquad r = 0.994.$$

   The peaks are separated by 0.004, exceeding the Fourier grid spacing $1/4096$, but smaller than the multitaper smoothing width $2W = 2(24)/4096 = 0.01171875$. I compute the periodogram and average 47 unit-norm DPSS estimates with $NW = 24$:

```
tapers = dpss(n, NW=24, Kmax=47, norm=2)
f, raw = periodogram(sample)
f, mt = multitaper_spectrum(sample, tapers)
```

   The periodogram resolves the two clusters near the true peaks. The multitaper estimate averages across both peaks and produces one broad feature. This is local smoothing bias, not a failure of the simulation.

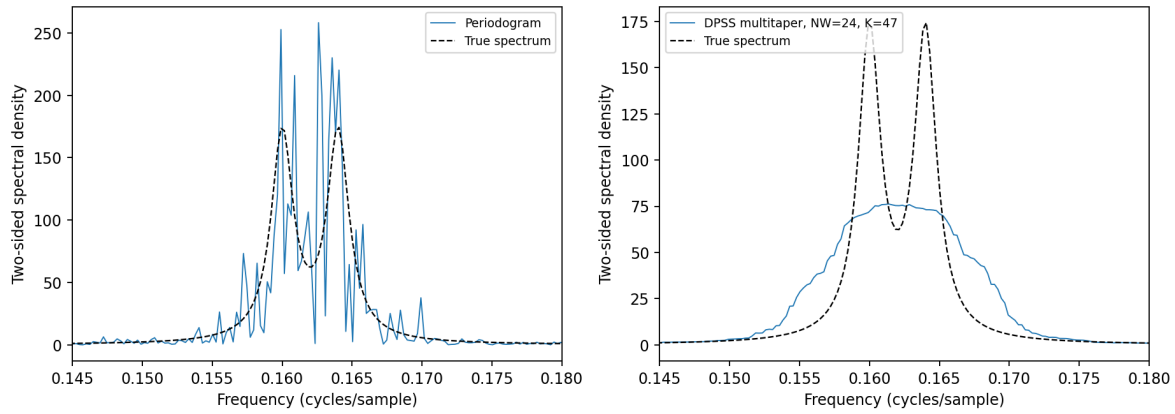


Figure 4: The same realization analyzed with the periodogram and a wide-band DPSS multitaper. Dashed curves are the true spectrum.

   (b) The periodogram loses a faint spectral peak, but the multitaper catches it.

   Sol.

   I construct a broad but faint peak on a white-noise floor, together with a strong peak elsewhere.

Define the periodized Gaussian peak

$$G_{v,c,b}(f) = \frac{v}{2\sqrt{2\pi}b} \sum_{j\in\mathbb{Z}} \left[e^{-(f-c+j)^2/(2b^2)} + e^{-(f+c+j)^2/(2b^2)} \right].$$

Its autocovariance is $ve^{-2\pi^2 b^2 h^2}\cos(2\pi ch)$. I use

$$S(f) = 1 + G_{2,0.10,0.004}(f) + G_{0.02,0.27,0.006}(f), \qquad n = 4096.$$

The peak at 0.27 rises only from a background near 1 to approximately 1.665. The unaveraged periodogram fluctuates enough that it is difficult to distinguish this broad feature from unrelated spikes. With $NW = 8$ and 15 DPSS tapers, the variance is reduced and the weak elevated band becomes visible. The half-bandwidth $8/4096$ is small relative to the weak peak's width, so smoothing is useful here rather than merging distinct features.

For 128 additional independent simulations, at the grid point nearest 0.27, the periodogram had mean 1.8439 and standard deviation 1.5298; the multitaper had mean 1.5616 and standard deviation 0.4146. These are Monte Carlo summaries, not exact expectations. The multitaper reduces noise substantially at the price of modest downward peak bias. Neither estimator is guaranteed to detect a weak peak in every realization.

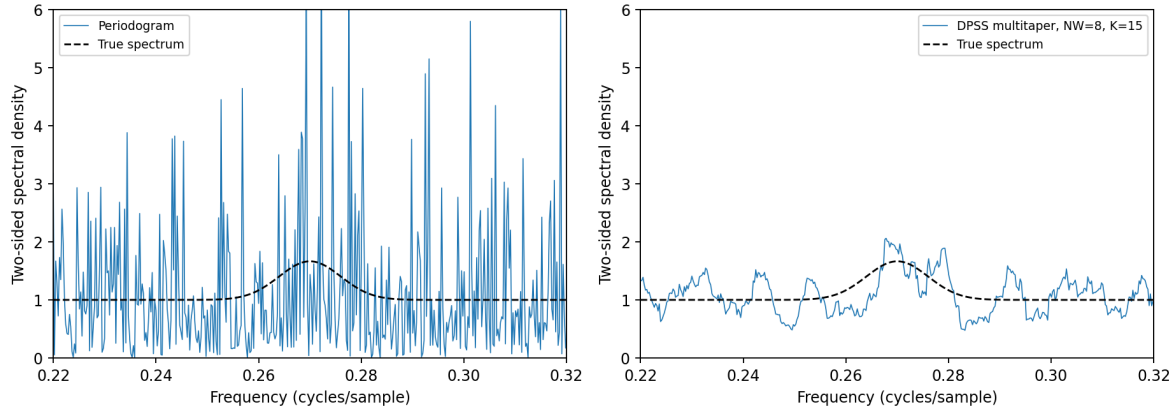


Figure 5: Weak spectral feature around 0.27 on the same data. Both panels use a common vertical range to make variance reduction visible.

R functions like `spec.pgram` and `spec.mtm`, the latter being from the `multitaper` package, may be helpful. In both cases, you have multiple options for achieving this. You could design an ARMA process, but you may find it easier to design the *spectrum* that makes the pathology obvious and then back out the ACF from it. *You are welcome to use chatbots to solve hard integrals. But please be disciplined about restricting your use of them to just the hard integrals!*

3. A *harmonic* of a fundamental frequency f_0 is an integer multiple kf_0 for $k \in \{2, 3, 4, \ldots\}$. Sometimes simple nonlinear transformations or other irregularities can cause your SDF estimator to also have peaks at these harmonics of "true" peaks. Let $n = 2048$ or something like that, and consider the time series

$$X_t = \tanh\big(\alpha\cos(2\pi f_0 t + \phi)\big) + Z_t,$$

where Z_t is a white noise with variance σ^2, $f_0 = 0.05$, and $\phi \sim \text{Unif}(0, 2\pi)$. Simulate such an X_t. Does it look like it could be a stationary process? Look at a spectral density estimator for it as if it were, and play with values of α to make the harmonics more or less prominent. Then find another nonlinear function besides tanh to apply to your periodic function that also induces spurious harmonics in your estimator. Try to pick something interesting that gives you stationary-looking data!

Sol.

I use $n = 2048$, $f_0 = 0.05$, independent Gaussian noise with $\sigma = 0.2$, and one uniform phase shared by the whole realization. The code for the transformed signal is

```
phase = rng.uniform(0, 2 * np.pi)
wave = np.cos(2 * np.pi * f0 * np.arange(n) + phase)
series = np.tanh(alpha * wave) + noise
```

With an independent i.i.d. noise process, this construction is strictly stationary: shifting all times replaces the uniform phase by another uniform phase modulo 2π. A plot alone is not needed to establish this.

For small α, $\tanh(\alpha \cos u) \approx \alpha \cos u$ and the fundamental dominates. As α increases, saturation makes the waveform closer to a square wave and odd harmonics appear. Half-period antisymmetry excludes even harmonics in the noiseless signal. The DPSS estimates at the third harmonic were approximately 0.0678, 3.9716, and 15.8890 for $\alpha = 0.5, 2, 8$, respectively; the noise floor is about 0.04.

As another nonlinearity I use half-wave rectification, $g(u) = \max(u, 0)$. Since

$$g(\cos u) = \tfrac{1}{2} \cos u + \tfrac{1}{2} |\cos u|,$$

it produces a nonzero constant, the fundamental, and even harmonics. After subtracting the sample mean, the simulation shows the fundamental near 0.05 and peaks near 0.10 and 0.20. Random phase again ensures stationarity, even though individual trajectories look periodic.

Strictly, these stationary random-phase processes have spectral atoms plus the white-noise density. The plotted estimates smooth the atoms into peaks; the periodic component does not have an ordinary Lebesgue spectral density. Frequencies above Nyquist are aliased.

4. Pull the tide gauge data from Canvas. Pick a segment of moderate length (at least a few thousand points) with no gaps, and compute several spectral density estimators: a periodogram, a sine multitaper, a DPSS multitaper, and whatever else your heart desires. Inspect each of them and try to answer: how many of these peaks are real, and how many are harmonics? How would you model this spectral density? You might consider taking a look at a list of lunar frequencies, which you can find on, e.g., Wikipedia. You don't have to include an answer to this part of the question, but try to ponder: how might you distinguish that non-linear mean from structure in a stationary covariance function?

Sol.

I use the supplied `tidegauge.csv`, which has date fields `year, month, day, hour` and observations in `actual`. All 119,266 rows have finite observations and consecutive hourly timestamps. I

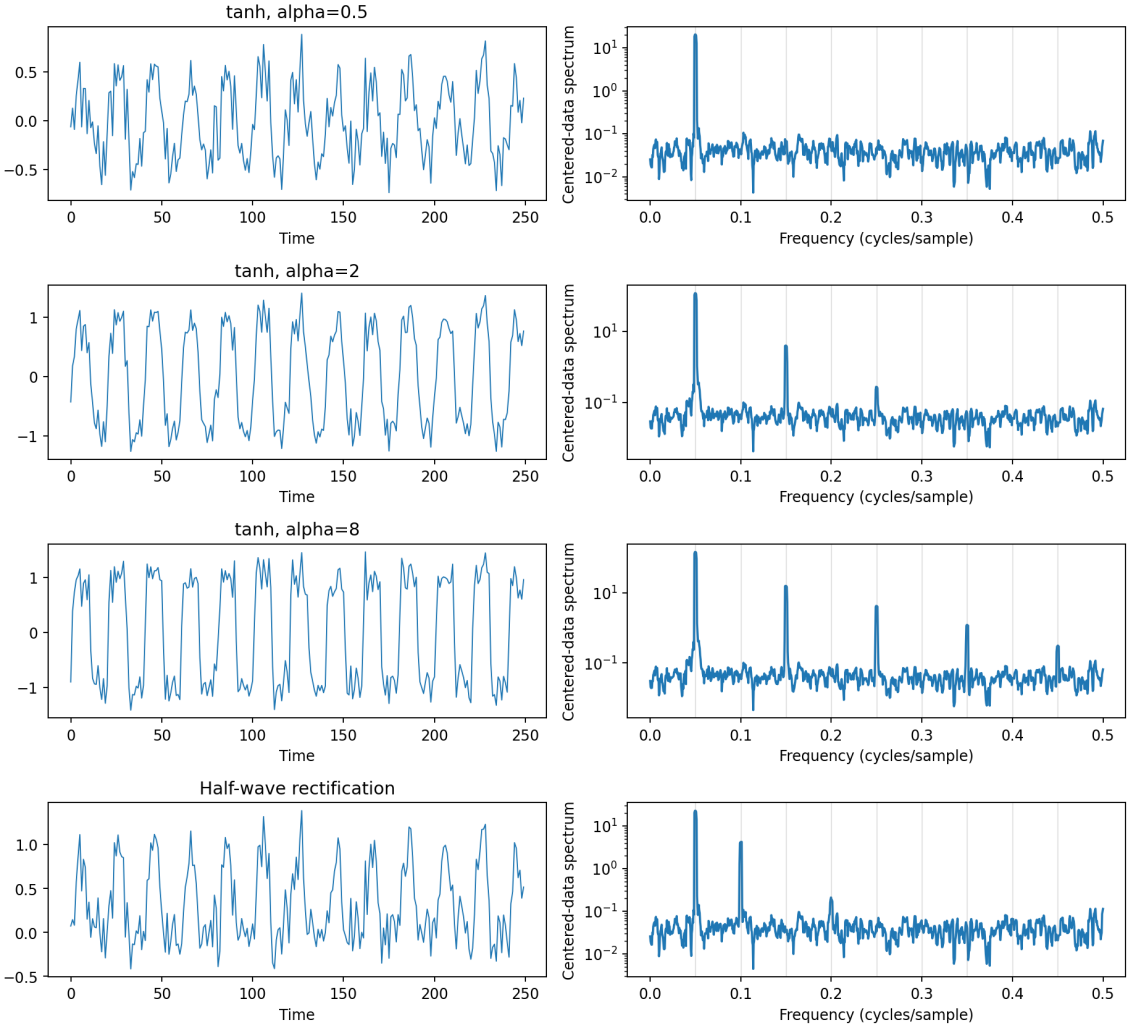


Figure 6: Nonlinear random-phase signals and centered-data DPSS estimates ($NW = 3$, five tapers). Vertical lines mark multiples of f_0.

select the first 8192 observations, from October 1, 1980 at 13:00 to September 7, 1981 at 20:00, without interpolation. The frequency grid spacing is $24/8192 = 0.00292969$ cycles per day; the DPSS half-bandwidth with $NW = 4$ is 0.01171875 cycles per day.

The code compares the ordinary periodogram, seven sine tapers, and seven unit-norm DPSS tapers. It subtracts the segment mean before computing spectra:

```
time = pd.to_datetime(table[["year", "month", "day", "hour"]])
values = table["actual"].to_numpy()
series = values[:8192] - values[:8192].mean()
f, raw = periodogram(series)
f, sine = multitaper_spectrum(series, sine_tapers(8192, 7))
f, mt = multitaper_spectrum(
    series, dpss(8192, NW=4, Kmax=7, norm=2)
)
```

For conversion from cycles per hourly observation to cycles per day I multiply frequency by 24 and divide density by 24, preserving integrated spectral power. The periodogram resolves sharper lines but fluctuates more; both multitaper estimates give smoother peaks with wider tops.

There are at least five prominent astronomical peak groups, around 0.93, 1.00, 1.90, 1.93, and 2.00 cycles per day. The strongest diurnal and semidiurnal components are consistent with the standard constituent frequencies below. The raw-grid peaks are approximate locations, not new estimates precise to all the quoted reference digits.

Constituent	Reference frequency (cycles/day)	Nearby raw peak
O_1	0.92954	0.92871
K_1	1.00274	1.00195
N_2	1.89598	1.89551
M_2	1.93227	1.93359
S_2	2.00000	2.00098

The reference speeds and constituent descriptions follow the NOAA harmonic-constituent table; only its universal frequencies are used, not another station's amplitudes. The P_1/K_1 and S_2/K_2 separations are smaller than the multitaper smoothing width, so these should be treated as unresolved groups in that plot. Smaller lunar constituents, such as Q_1, can also contribute.

A peak near twice a diurnal frequency need not be its harmonic: M_2 and S_2 are distinct astronomical forcing constituents. Conversely, weaker features near 3.8643 and 3.9229 cycles per day are compatible with the *overtide* M_4 at $2f_{M_2} = 3.86455$ and the compound constituent MS_4 at $f_{M_2} + f_{S_2} = 3.93227$, within the smoothing resolution. Other weak features occur near 2.8975, 3.0059, and 4.0049. These are candidates for higher or compound harmonics, but the plot alone cannot assign every one uniquely. They can be physical nonlinear tidal signals rather than estimation artifacts. Thus counting every local maximum as an independent oscillation would overstate what the data resolve; there is no unambiguous single count of "real" versus "spurious" peaks from these estimates alone.

A useful model is a deterministic harmonic mean plus a stochastic residual:

$$X_t = \mu + \beta t + \sum_j \{a_j \cos(2\pi f_j t) + b_j \sin(2\pi f_j t)\} + \varepsilon_t,$$

where t is measured in days. I fit an intercept, a linear trend, eight principal astronomical constituents ($Q_1, O_1, P_1, K_1, N_2, M_2, S_2, K_2$), and three overtide/compound candidates (M_4, MS_4, M_6) by least squares. This descriptive fit explains 91.58% of the centered segment variance; its residual mean square is 0.002695 in squared supplied-data units. The dominant spectral peaks fall substantially after regression, although some narrow components and low-frequency residual variation remain. A smooth colored spectrum or an ARMA model for ε_t, with any remaining reproducible lines explicitly modelled, would be a reasonable next step.

The variance explained is in-sample, not a prediction score. To distinguish a nonlinear mean from stationary covariance structure, I would check phase coherence relative to astronomical forcing across disjoint windows and validate predictions and residuals on held-out periods. A stationary random-phase process can have the same spectral lines, so second-order spectral information alone cannot settle this distinction.

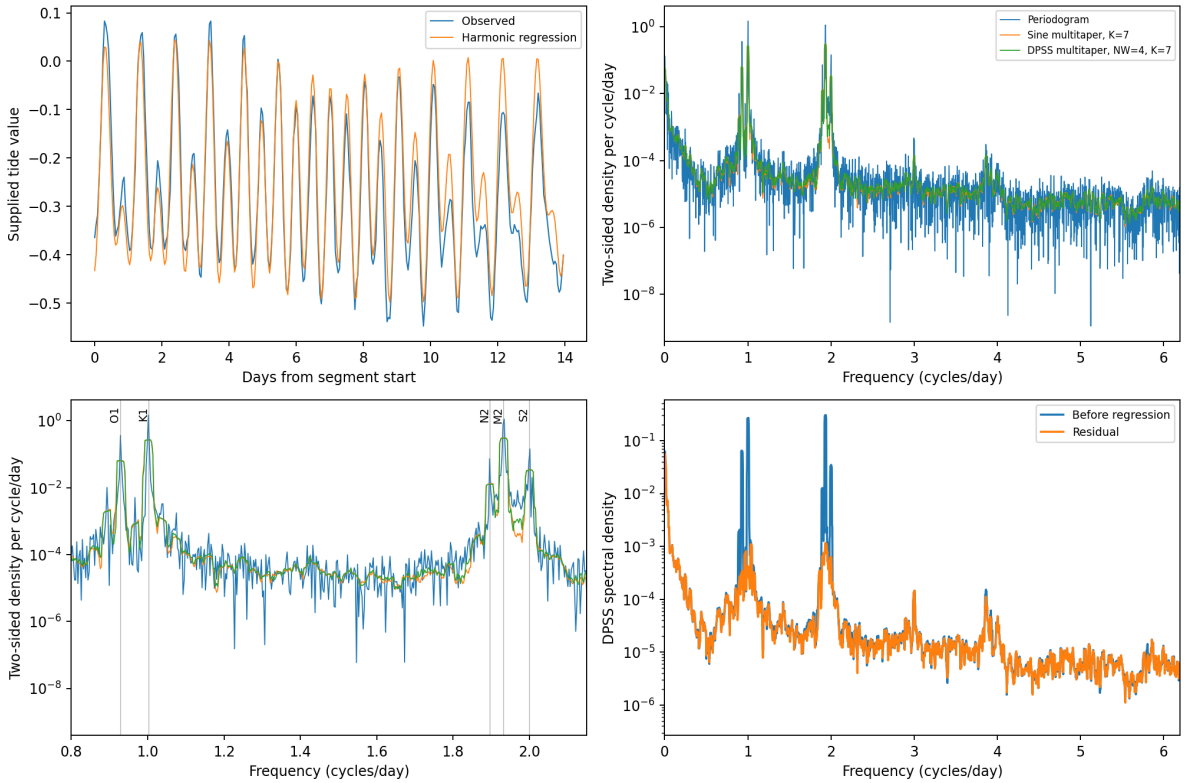


Figure 7: Observed hourly tide data, spectral estimates, and harmonic-regression residuals for the selected gap-free 8192-point segment.

Problem 3

Write code to implement the *Whittle likelihood* described in class. In particular, please write a function that takes two arguments: the (potentially weighted) DFT of your data, and SDF parameters.

1. Pull up the NAO data and take the first 1024 measurements. Look at an exploratory estimator for the spectrum (perhaps a multitaper…) and find a functional form you like for the spectrum. This should be something involving parameters, like $S_\theta(\omega) = \theta_0 e^{-\theta_1|\omega|}$ or something. Discuss your choices.

   Sol.

   I use the first 1024 daily measurements without gaps and subtract their sample mean, $\bar{X} = -0.34727$. As in HW2, the file is named `nao.csv`, but its supplied value column is labelled `aao_index_cdas`; I analyze the supplied values without relabelling them as a different downloaded index.

   The periodogram and a seven-taper DPSS estimate ($NW = 4$) show strong low-frequency power with an overall declining envelope and a small high-frequency floor. I choose the three-parameter family

   $$S_\theta(f) = ae^{-b|f|} + c, \qquad -\tfrac{1}{2} \le f \le \tfrac{1}{2}, \qquad a, b, c > 0.$$

   This is a simple positive even spectrum: a controls the excess amplitude, b controls its decay, and c represents a white-noise floor. It captures the broad shape without treating every noisy periodogram spike as a separate physical component. It is a deliberately parsimonious working model, not a claim that the true spectrum must be monotone.

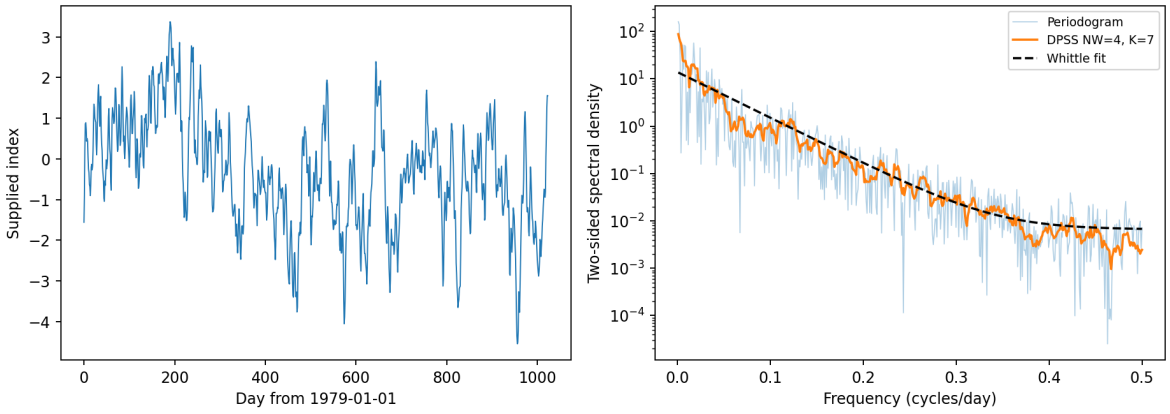


Figure 8: First 1024 supplied daily measurements and the fitted spectral shape, using two-sided density in cycles per day.

2. By specifying your Whittle likelihood and handing that function to an optimizer (be careful about bound constraints for valid parameters!), fit that functional form to the data.

   Sol.

   I compute $d_k = n^{-1/2} \sum_{t=0}^{n-1} (X_t - \bar{X}) e^{-2\pi i k t/n}$ and $I_k = |d_k|^2$. For real data I use only the positive

frequencies, with the real-valued Nyquist coefficient given half weight:

$$Q(\theta) = \sum_{k=1}^{n/2-1} \left[\log S_\theta(k/n) + \frac{I_k}{S_\theta(k/n)}\right] + \frac{1}{2}\left[\log S_\theta(1/2) + \frac{I_{n/2}}{S_\theta(1/2)}\right].$$

The zero frequency is excluded after estimating the mean. The DFT time origin differs from Problem 1 only by a unit-modulus phase and therefore does not change the periodogram. The two-sided spectrum is not doubled when only positive frequencies are used in this likelihood.

```
dft = np.fft.fft(series) / np.sqrt(len(series))
parameters, result = fit_spectrum(dft)
```

I optimize the log parameters with bounded L-BFGS-B and four initial decay values. All fitted parameters are positive; the selected optimizer reports convergence. The fitted spectrum is

$$\widehat{S}(f) = 13.89572e^{-22.24673|f|} + 0.00658014,$$

with objective $Q = -638.94618$, up to the omitted parameter-independent likelihood constants. This formula uses cycles per day; coefficients cannot be transferred unchanged to an angular-frequency density. For weighted DFT inputs the same function is a Whittle-type approximation; tapering smooths the expected periodogram and correlates nearby coefficients. The fit reported here uses the untapered DFT to keep this approximation clear.

3*. Using a numerical integrator or a chatbot or however you please, get from the spectral density to obtaining the corresponding ACF values $\{\gamma(0), \ldots, \gamma(n-1)\}$. Now take your fitted model SARIMA model from last week and evaluate the true likelihood, involving the covariance matrix and stuff, for each fitted model. Which of the two models has a better likelihood? Try and investigate what you think is going on and why one or the other was better.

Sol.

Fourier inversion gives an analytic autocovariance. For integer h,

$$\begin{aligned}\gamma_\theta(h) &= 2a\int_0^{1/2} e^{-bf}\cos(2\pi hf)\,df + c\,1_{\{h=0\}} \\ &= \frac{2ab\{1-(-1)^h e^{-b/2}\}}{b^2+(2\pi h)^2} + c\,1_{\{h=0\}}.\end{aligned}$$

Numerical quadrature at lags $0, 1, 10, 100, 1023$ agreed with this expression to better than 10^{-7}. Let $\Gamma_{ij} = \gamma_\theta(i-j)$. The exact Gaussian log likelihood for centered observations is

$$\ell = -\frac{1}{2}\{n\log(2\pi) + \log\det\Gamma + x^\top\Gamma^{-1}x\}.$$

The code uses a Cholesky factor and triangular solves rather than an explicit inverse. For the 1024-point vector, the fitted stationary model gives $\ell = -228.49829$, treating the sample-mean estimate as a fixed plug-in value.

The previous HW2 fit was

$$(1 - .6642B - .3518B^2 + .2324B^3 - .0837B^4)(1 - B^{365})X_t = (1 + 1.1556B + .3247B^2)Z_t,$$

with innovation variance 0.1956. This SARIMA model has a seasonal unit root, so it has no stationary covariance matrix for the raw levels. Comparing a raw-data stationary likelihood against a likelihood

of differenced data would use different observations and be misleading. Instead I evaluate both fitted models on the same 659-dimensional vector $Y_t = X_{t+365} - X_t$. For the stationary spectral model,

$$\gamma_Y(h) = 2\gamma(h) - \gamma(h + 365) - \gamma(h - 365).$$

For the SARIMA model, Y is the stationary ARMA$(4, 2)$ process above. Using the rounded coefficients reported in HW2, the exact marginal Gaussian log likelihoods of this common data vector are

$$\ell_{\text{spectral}}(Y) = -344.26496, \qquad \ell_{\text{SARIMA}}(Y) = -366.31558.$$

Thus the spectral model has the larger likelihood by 22.05062 on this common differenced sample. These are marginal likelihoods of the *differences*, not unconditional likelihoods for the original levels. A conditional-level comparison would additionally require specifying how the SARIMA innovations relate to the initial seasonal levels.

This result is consistent with the weak annual dependence noted in HW2: forcing annual integration may be a poor description of this segment. The comparison is descriptive, not an out-of-sample test: the old coefficients were fitted on the larger record, whereas the spectral model was fitted to these first 1024 observations, and the models have different complexity. Nor is the new model perfect. Its variance is 1.25580, compared with the centered sample variance 1.87984, and its ACF misses the slower empirical component. Whittle diagonalization, finite-window effects and spectral misspecification can all contribute to this discrepancy.

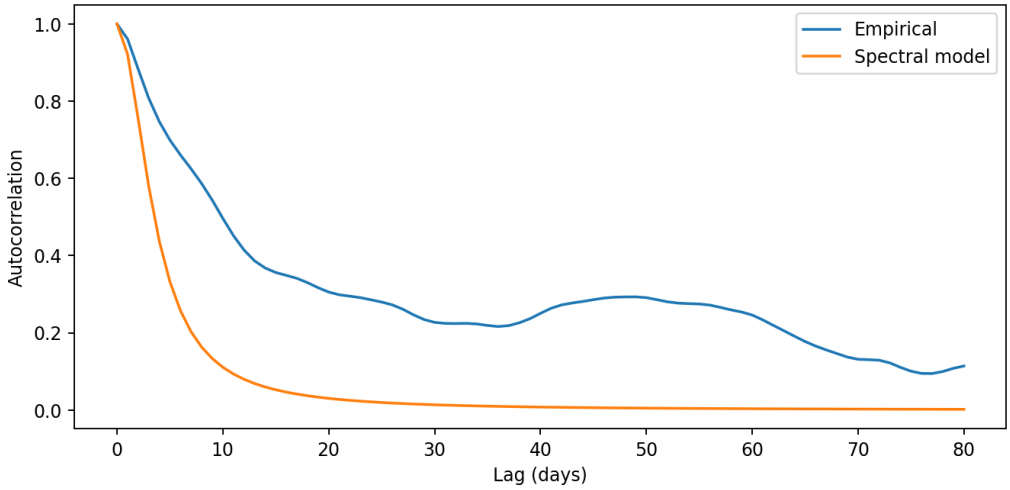


Figure 9: Empirical and fitted autocorrelation functions. The simple fitted spectrum does not reproduce all long-lag dependence.

A Python code

The following scripts reproduce the simulations, estimates, and figures. Data are stored in `data/`; figures are written to `png/`. The supplied tide data are included in `data/tidegauge.csv`. The scripts contain no comments. All spectral estimates use the two-sided density convention even when only positive frequencies are displayed.

A.1 Problem 1

```
from pathlib import Path
from statsmodels.tsa.arima_process import arma_generate_sample
from scipy.signal.windows import dpss

import json
from scipy.linalg import toeplitz

import numpy as np
import matplotlib.pyplot as plt

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
FIGURE_DIR = ROOT / "png"
SEED = 70103

N = 512
M = 10 * N
NW = 10


def simulated_paths(times_series, alpha = 10, rng=None):
    n = times_series.shape[0]
    weights = np.ones(n)/(n ** 0.5)
    frequencies = np.arange(-np.floor((n-1)/2),np.floor(n/2)+1)/n
    dft = np.stack([weighted_dft(times_series, weights, frequency) for frequency in frequencies])
    return dft

def simulated_paths_2(times_series, alpha = 10, rng=None):
    n = times_series.shape[0]
    weights = dpss(n, NW=10, Kmax=1, norm=2)[0]
    frequencies = np.arange(-np.floor((n-1)/2),np.floor(n/2)+1)/n
    dft = np.stack([weighted_dft(times_series, weights, frequency) for frequency in frequencies])
    return dft

def draw_paths(paths, x=None, component="real", ax=None):
    paths = np.asarray(paths)
    if paths.ndim != 2 or 0 in paths.shape:
        raise ValueError("paths must be a nonempty (m, n) array")
    if component == "real":
        values = paths.real
    elif component == "imag":
        values = paths.imag
    elif component == "abs":
```

```
        values = np.abs(paths)
    else:
        raise ValueError("component must be 'real', 'imag', or 'abs'")

    m, n = paths.shape
    x = np.arange(1, n + 1) if x is None else np.asarray(x)
    if x.ndim != 1 or x.size != n:
        raise ValueError("x must be a one-dimensional array of length n")
    if ax is None:
        _, ax = plt.subplots(figsize=(10, 5))

    colors = plt.get_cmap("viridis")(np.linspace(0.05, 0.95, m))
    lines = ax.plot(x, values.T, linewidth=0.9, alpha=0.8 if m <= 20 else 0.25)
    for index, (line, color) in enumerate(zip(lines, colors), start=1):
        line.set_color(color)
        line.set_label(f"Series {index}")
    ax.set_xlabel("Index")
    ax.set_ylabel({"real": "Real part", "imag": "Imaginary part", "abs": "Magnitude"}[component])
    ax.grid(alpha=0.2)
    if m <= 10:
        ax.legend()
    return ax



def draw_matrix(matrix, component="abs", ax=None, cmap=None, colorbar=True, gamma=0.4):
    from matplotlib.colors import PowerNorm

    if not np.isfinite(gamma) or gamma <= 0:
        raise ValueError("gamma must be a positive finite number")
    matrix = np.asarray(matrix)
    if matrix.ndim != 2 or 0 in matrix.shape:
        raise ValueError("matrix must be a nonempty two-dimensional array")
    if component == "abs":
        values = np.abs(matrix)
    elif component == "real":
        values = matrix.real
    elif component == "imag":
        values = matrix.imag
    else:
        raise ValueError("component must be 'abs', 'real', or 'imag'")
    if not np.isfinite(values).all():
        raise ValueError("matrix values must be finite")
    if ax is None:
        _, ax = plt.subplots(figsize=(8, 6))

    options = {}
    if component == "abs":
        cmap = "viridis" if cmap is None else cmap
        limit = float(np.max(values)) or 1.0
        options["norm"] = PowerNorm(gamma=gamma, vmin=0, vmax=limit)
    else:
        cmap = "RdBu_r" if cmap is None else cmap
        limit = float(np.max(np.abs(values))) or 1.0
```

```
        options.update(vmin=-limit, vmax=limit)
    image = ax.imshow(values, origin="upper", interpolation="nearest",
                      aspect="auto", cmap=cmap, **options)
    ax.set_xlabel("Column index")
    ax.set_ylabel("Row index")
    if colorbar:
        bar = ax.figure.colorbar(image, ax=ax)
        bar.set_label({"abs": "Magnitude", "real": "Real part", "imag": "Imaginary part"}[component])
    return ax


def weighted_dft(paths, weights, frequency):
    time = np.arange(1,paths.shape[0]+1)
    exp_term = np.exp( - 1j * 2 * np.pi * frequency * time)
    return np.sum(weights * paths * exp_term)


def empirical_dft_covariance(coefficients):
    dft_covaiance = np.outer(coefficients[0], coefficients[0].conj())
    for coefficient in coefficients[1:]:
        dft_covaiance += np.outer(coefficient, coefficient.conj())
    return dft_covaiance


def batch_dft(samples, weights):
    n = samples.shape[1]
    k = np.arange(-((n - 1) // 2), n // 2 + 1)
    frequencies = k / n
    coefficients = np.fft.fft(samples * weights, axis=1)[:, k % n]
    return frequencies, coefficients * np.exp(-2j * np.pi * frequencies)


def covariance_correlation(matrix):
    standard_deviations = np.sqrt(np.diag(matrix).real)
    return matrix / np.outer(standard_deviations, standard_deviations)


def main():
    FIGURE_DIR.mkdir(parents=True, exist_ok=True)
    rng = np.random.default_rng(SEED)
    samples = np.stack([arma_generate_sample(ar=[1, -0.6], ma=[1, 0.3], nsample=N,
                        burnin=500, distrvs=rng.standard_normal) for _ in range(M)])
    ax = draw_paths(samples[:10])
    ax.set_title("ARMA(1,1) sample paths")
    ax.set_xlabel("Time")
    ax.figure.savefig(FIGURE_DIR / "sample_paths.png", dpi=170, bbox_inches="tight")
    plt.close(ax.figure)
    weights = {"flat": np.ones(N) / np.sqrt(N), "dpss": dpss(N, NW=NW, Kmax=1, norm=2)[0]}
    h = np.arange(N)
    acf = np.empty(N)
    acf[0] = (1 + 0.3 ** 2 + 2 * 0.6 * 0.3) / (1 - 0.6 ** 2)
    acf[1:] = (0.6 + 0.3) * (1 + 0.6 * 0.3) / (1 - 0.6 ** 2) * 0.6 ** (h[1:] - 1)
    results = {"n": N, "m": M, "ar": 0.6, "ma": 0.3, "gamma0": float(acf[0]), "gamma1": float(acf[1])}
    fig, axes = plt.subplots(2, 2, figsize=(11, 9))
```

```
    for row, (name, taper) in enumerate(weights.items()):
        frequencies, coefficients = batch_dft(samples, taper)
        direct = np.array([weighted_dft(samples[0], taper, frequency) for frequency in frequencies])
        np.testing.assert_allclose(coefficients[0], direct, atol=1e-11)
        covariance = empirical_dft_covariance(coefficients) / M
        kernel = taper[None, :] * np.exp(-2j * np.pi * frequencies[:, None] * np.arange(1, N + 1))
        expected = kernel @ toeplitz(acf) @ kernel.conj().T
        empirical_correlation = covariance_correlation(covariance)
        theoretical_correlation = covariance_correlation(expected)
        ax = draw_matrix(covariance)
        ax.set_title(f"ARMA(1,1) DFT covariance: {name}")
        filename = "empirical_covariance_matrix.png" if name == "flat" else "
    empirical_covariance_matrix_dpss.png"
        ax.figure.savefig(FIGURE_DIR / filename, dpi=170, bbox_inches="tight")
        plt.close(ax.figure)
        for column, (matrix, label) in enumerate(((empirical_correlation, "Empirical"), (
    theoretical_correlation, "Theoretical"))):
            image = axes[row, column].imshow(np.abs(matrix), origin="upper", vmin=0, vmax=1, cmap="viridis
    ")
            axes[row, column].set_title(f"{name}: {label} correlation magnitude")
            axes[row, column].set_xlabel("Frequency index")
            axes[row, column].set_ylabel("Frequency index")
            fig.colorbar(image, ax=axes[row, column])
        off_diagonal = expected - np.diag(np.diag(expected))
        results[name] = {"relative_diagonal_approximation_error": float(np.linalg.norm(off_diagonal) / np.
    linalg.norm(expected)),
                         "mean_adjacent_correlation": float(np.mean(np.abs(np.diag(theoretical_correlation
    , 1)))),
                         "empirical_mean_adjacent_correlation": float(np.mean(np.abs(np.diag(
    empirical_correlation, 1)))),
                         "relative_simulation_error": float(np.linalg.norm(covariance - expected) / np.
    linalg.norm(expected))}
    fig.tight_layout()
    fig.savefig(FIGURE_DIR / "problem1_correlation_comparison.png", dpi=170, bbox_inches="tight")
    plt.close(fig)
    (ROOT / "problem1_results.json").write_text(json.dumps(results, indent=2))
    print(json.dumps(results, indent=2))


if __name__ == "__main__":
    main()
```

A.2 Problem 2

```python
from pathlib import Path
import json

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.signal.windows import dpss
from scipy.signal import find_peaks

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
FIGURE_DIR = ROOT / "png"
SEED = 70103
TIDE_DATA_PATH = DATA_DIR / "tidegauge.csv"


def circulant_embedding(acf, rng=None):
    acf = np.asarray(acf, dtype=float)
    if acf.ndim != 1 or len(acf) == 0 or not np.isfinite(acf).all():
        raise ValueError("acf must be a finite nonempty vector")
    rng = np.random.default_rng() if rng is None else rng
    if len(acf) == 1:
        if acf[0] < 0:
            raise ValueError("negative variance")
        return rng.normal(scale=np.sqrt(acf[0]), size=1)
    row = np.r_[acf, acf[-2:0:-1]]
    eigenvalues = np.fft.fft(row).real
    tolerance = 1e-11 * max(1.0, np.max(np.abs(eigenvalues)))
    if eigenvalues.min() < -tolerance:
        raise ValueError("Embedding is not positive semidefinite; provide a valid longer covariance
     extension")
    eigenvalues = np.maximum(eigenvalues, 0)
    white_noise = rng.standard_normal(len(row))
    sample = np.fft.ifft(np.sqrt(eigenvalues) * np.fft.fft(white_noise)).real
    return sample[:len(acf)]


def periodogram(series):
    series = np.asarray(series, dtype=float)
    return np.fft.rfftfreq(len(series)), np.abs(np.fft.rfft(series)) ** 2 / len(series)


def multitaper_spectrum(series, tapers):
    series = np.asarray(series, dtype=float)
    tapers = np.atleast_2d(tapers)
    if tapers.shape[1] != len(series):
        raise ValueError("tapers must have the same length as the data")
    estimates = np.abs(np.fft.rfft(tapers * series, axis=1)) ** 2
    estimates /= np.sum(tapers ** 2, axis=1)[:, None]
    return np.fft.rfftfreq(len(series)), estimates.mean(axis=0)


```

```python
def sine_tapers(n, k):
    t = np.arange(1, n + 1)
    orders = np.arange(1, k + 1)
    return np.sqrt(2 / (n + 1)) * np.sin(np.pi * orders[:, None] * t / (n + 1))


def normal_peak_acf(n, variance, center, width):
    h = np.arange(n)
    return variance * np.exp(-2 * np.pi ** 2 * width ** 2 * h ** 2) * np.cos(2 * np.pi * center * h)


def normal_peak_spectrum(frequencies, variance, center, width):
    output = np.zeros_like(frequencies, dtype=float)
    for shift in (-1, 0, 1):
        for sign in (-1, 1):
            output += variance / (2 * np.sqrt(2 * np.pi) * width) * np.exp(-0.5 * ((frequencies - sign * center + shift) / width) ** 2)
    return output


def save_figure(fig, name):
    fig.tight_layout()
    fig.savefig(FIGURE_DIR / name, dpi=170, bbox_inches="tight")
    plt.close(fig)


def compare_peak_resolution(rng):
    n = 4096
    h = np.arange(n)
    centers = (0.16, 0.164)
    r = 0.994
    acf = sum(r ** h * np.cos(2 * np.pi * f * h) for f in centers)
    acf[0] += 0.05
    sample = circulant_embedding(acf, rng)
    f, raw = periodogram(sample)
    tapers = dpss(n, NW=24, Kmax=47, norm=2)
    _, mt = multitaper_spectrum(sample, tapers)
    truth = np.full_like(f, 0.05)
    for center in centers:
        for sign in (-1, 1):
            truth += 0.5 * (1 - r ** 2) / (1 + r ** 2 - 2 * r * np.cos(2 * np.pi * (f - sign * center)))
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
    for ax, estimate, label in zip(axes, (raw, mt), ("Periodogram", "DPSS multitaper, NW=24, K=47")):
        ax.plot(f, estimate, linewidth=0.8, label=label)
        ax.plot(f, truth, "k--", linewidth=1, label="True spectrum")
        ax.set_xlim(0.145, 0.18)
        ax.set_xlabel("Frequency (cycles/sample)")
        ax.set_ylabel("Two-sided spectral density")
        ax.legend(fontsize=8)
    save_figure(fig, "problem2_resolution.png")
    return {"n": n, "centers": centers, "r": r, "NW": 24, "K": 47, "bandwidth": 48 / n}


def compare_faint_peak_detection(rng):
```

```
    n = 4096
    components = ((2.0, 0.10, 0.004), (0.02, 0.27, 0.006))
    acf = sum(normal_peak_acf(n, *component) for component in components)
    acf[0] += 1.0
    sample = circulant_embedding(acf, rng)
    f, raw = periodogram(sample)
    tapers = dpss(n, NW=8, Kmax=15, norm=2)
    _, mt = multitaper_spectrum(sample, tapers)
    truth = np.ones_like(f)
    for component in components:
        truth += normal_peak_spectrum(f, *component)
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
    for ax, estimate, label in zip(axes, (raw, mt), ("Periodogram", "DPSS multitaper, NW=8, K=15")):
        ax.plot(f, estimate, linewidth=0.7, label=label)
        ax.plot(f, truth, "k--", linewidth=1.2, label="True spectrum")
        ax.set_xlim(0.22, 0.32)
        ax.set_ylim(0, 6)
        ax.set_xlabel("Frequency (cycles/sample)")
        ax.set_ylabel("Two-sided spectral density")
        ax.legend(fontsize=8)
    save_figure(fig, "problem2_faint_peak.png")
    center_index = np.argmin(np.abs(f - 0.27))
    raw_values, mt_values = [], []
    for _ in range(128):
        realization = circulant_embedding(acf, rng)
        raw_values.append(periodogram(realization)[1][center_index])
        mt_values.append(multitaper_spectrum(realization, tapers)[1][center_index])
    return {"n": n, "components": components, "noise_variance": 1.0, "NW": 8, "K": 15,
            "true_peak": float(truth[center_index]), "raw_mean": float(np.mean(raw_values)),
            "raw_sd": float(np.std(raw_values, ddof=1)), "mt_mean": float(np.mean(mt_values)),
            "mt_sd": float(np.std(mt_values, ddof=1)), "replicates": 128}


def explore_harmonics(rng):
    n = 2048
    t = np.arange(n)
    f0 = 0.05
    phase = rng.uniform(0, 2 * np.pi)
    wave = np.cos(2 * np.pi * f0 * t + phase)
    noise = rng.normal(scale=0.2, size=n)
    transformations = [("tanh, alpha=0.5", np.tanh(0.5 * wave)),
                       ("tanh, alpha=2", np.tanh(2 * wave)),
                       ("tanh, alpha=8", np.tanh(8 * wave)),
                       ("Half-wave rectification", np.maximum(wave, 0))]
    tapers = dpss(n, NW=3, Kmax=5, norm=2)
    fig, axes = plt.subplots(4, 2, figsize=(11, 10))
    results = {}
    for row, (label, transformed) in enumerate(transformations):
        series = transformed + noise
        f, estimate = multitaper_spectrum(series - series.mean(), tapers)
        axes[row, 0].plot(t[:250], series[:250], linewidth=0.8)
        axes[row, 0].set_title(label)
        axes[row, 0].set_xlabel("Time")
        axes[row, 1].semilogy(f, estimate)
```

```
        for multiple in range(1, 10):
            axes[row, 1].axvline(multiple * f0, color="gray", alpha=0.3, linewidth=0.6)
        axes[row, 1].set_xlabel("Frequency (cycles/sample)")
        axes[row, 1].set_ylabel("Centered-data spectrum")
        results[label] = [float(estimate[np.argmin(np.abs(f - multiple * f0))]) for multiple in range(1,
     6)]
    save_figure(fig, "problem2_harmonics.png")
    return results


def analyze_tide_data(path=TIDE_DATA_PATH):
    table = pd.read_csv(path)
    if {"year", "month", "day", "hour", "actual"}.issubset(table.columns):
        time = pd.to_datetime(table[["year", "month", "day", "hour"]], errors="raise")
        values = pd.to_numeric(table["actual"], errors="coerce").to_numpy()
    elif table.shape[1] >= 2:
        time = pd.to_datetime(table.iloc[:, 0], errors="raise")
        values = pd.to_numeric(table.iloc[:, 1], errors="coerce").to_numpy()
    else:
        raise ValueError("Provide date columns and actual, or timestamp and observation columns")
    delta = np.diff(time.astype("int64").to_numpy())
    positive = delta[delta > 0]
    if len(positive) == 0:
        raise ValueError("No positive sampling intervals")
    unique, counts = np.unique(positive, return_counts=True)
    spacing = unique[np.argmax(counts)]
    best_start, best_end, start = 0, 0, 0
    for end in range(len(values)):
        if not np.isfinite(values[end]):
            start = end + 1
        elif end > start and delta[end - 1] != spacing:
            start = end
        if end + 1 - start > best_end - best_start:
            best_start, best_end = start, end + 1
    if best_end - best_start < 3000:
        raise ValueError("No regularly sampled gap-free segment of at least 3000 points")
    values = values[best_start:min(best_end, best_start + 8192)]
    series = values - values.mean()
    n = len(series)
    estimators = {"Periodogram": periodogram(series),
                  "Sine multitaper, K=7": multitaper_spectrum(series, sine_tapers(n, 7)),
                  "DPSS multitaper, NW=4, K=7": multitaper_spectrum(series, dpss(n, 4, Kmax=7, norm=2))}
    samples_per_day = 86400e9 / spacing
    constituents = {"Q1": 13.3986609 / 15, "O1": 13.9430356 / 15,
                    "P1": 14.9589314 / 15, "K1": 15.0410686 / 15,
                    "N2": 28.4397295 / 15, "M2": 28.9841042 / 15,
                    "S2": 2.0, "K2": 30.0821373 / 15,
                    "M4": 57.9682084 / 15, "MS4": 58.9841042 / 15,
                    "M6": 86.9523126 / 15}
    days = np.arange(n) / samples_per_day
    columns = [np.ones(n), days - days.mean()]
    for frequency in constituents.values():
        columns.extend([np.cos(2 * np.pi * frequency * days), np.sin(2 * np.pi * frequency * days)])
    design = np.column_stack(columns)
```

```
    coefficients = np.linalg.lstsq(design, values, rcond=None)[0]
    fitted = design @ coefficients
    residual = values - fitted
    explained_fraction = 1 - np.sum(residual ** 2) / np.sum(series ** 2)
    amplitudes = {name: float(np.hypot(coefficients[2 + 2 * j], coefficients[3 + 2 * j]))
                  for j, name in enumerate(constituents)}
    fig, axes = plt.subplots(2, 2, figsize=(12, 8))
    show = min(n, int(samples_per_day * 14))
    axes[0, 0].plot(days[:show], values[:show], label="Observed", linewidth=0.9)
    axes[0, 0].plot(days[:show], fitted[:show], label="Harmonic regression", linewidth=0.8)
    axes[0, 0].set_xlabel("Days from segment start")
    axes[0, 0].set_ylabel("Supplied tide value")
    axes[0, 0].legend(fontsize=8)
    for label, (f, estimate) in estimators.items():
        for ax in (axes[0, 1], axes[1, 0]):
            ax.semilogy(f[1:] * samples_per_day, estimate[1:] / samples_per_day, label=label, linewidth
=0.75)
    axes[0, 1].set_xlim(0, 6.2)
    axes[1, 0].set_xlim(0.8, 2.15)
    for ax in (axes[0, 1], axes[1, 0]):
        ax.set_xlabel("Frequency (cycles/day)")
        ax.set_ylabel("Two-sided density per cycle/day")
    axes[0, 1].legend(fontsize=7)
    for name in ("O1", "K1", "N2", "M2", "S2"):
        axes[1, 0].axvline(constituents[name], color="gray", alpha=0.5, linewidth=0.7)
        axes[1, 0].text(constituents[name], 0.98, name, rotation=90,
                        transform=axes[1, 0].get_xaxis_transform(), ha="right", va="top", fontsize=8)
    f, residual_spectrum = multitaper_spectrum(residual, dpss(n, 4, Kmax=7, norm=2))
    tide_spectrum = estimators["DPSS multitaper, NW=4, K=7"][1]
    axes[1, 1].semilogy(f[1:] * samples_per_day, tide_spectrum[1:] / samples_per_day, label="Before
 regression")
    axes[1, 1].semilogy(f[1:] * samples_per_day, residual_spectrum[1:] / samples_per_day, label="Residual"
)
    axes[1, 1].set_xlim(0, 6.2)
    axes[1, 1].set_xlabel("Frequency (cycles/day)")
    axes[1, 1].set_ylabel("DPSS spectral density")
    axes[1, 1].legend(fontsize=8)
    save_figure(fig, "problem2_tide.png")
    peaks = find_peaks(tide_spectrum[1:], distance=8)[0] + 1
    peaks = peaks[np.argsort(tide_spectrum[peaks])[-15:][::-1]]
    raw = estimators["Periodogram"][1]
    peak_matches = {}
    for name, frequency in constituents.items():
        indices = np.flatnonzero(np.abs(f * samples_per_day - frequency) <= 2 * samples_per_day / n)
        index = indices[np.argmax(raw[indices])]
        peak_matches[name] = {"reference_frequency": frequency, "nearest_local_raw_peak": float(f[index] *
 samples_per_day),
                              "raw_peak_density": float(raw[index] / samples_per_day), "
regression_amplitude": amplitudes[name]}
    return {"n": n, "total_rows": len(table), "start": str(time.iloc[best_start]),
            "end": str(time.iloc[best_start + n - 1]), "gap_free_length": best_end - best_start,
            "samples_per_day": float(samples_per_day), "grid_spacing_cycles_per_day": float(
samples_per_day / n),
            "multitaper_half_bandwidth_cycles_per_day": float(4 * samples_per_day / n),
```

```
            "peaks_cycles_per_day": (f[peaks] * samples_per_day).tolist(),
            "peaks_density": (tide_spectrum[peaks] / samples_per_day).tolist(),
            "explained_fraction": float(explained_fraction), "residual_variance": float(np.mean(residual
     ** 2)),
            "constituents": peak_matches}


def main():
    FIGURE_DIR.mkdir(parents=True, exist_ok=True)
    rng = np.random.default_rng(SEED)
    results = {"resolution": compare_peak_resolution(rng),
               "faint_peak": compare_faint_peak_detection(rng),
               "harmonics": explore_harmonics(rng)}
    if TIDE_DATA_PATH.exists():
        results["tide"] = analyze_tide_data()
    else:
        results["tide"] = {"status": "Canvas tide data not supplied"}
    (ROOT / "problem2_results.json").write_text(json.dumps(results, indent=2))
    print(json.dumps(results, indent=2))


if __name__ == "__main__":
    main()
```

A.3 Problem 3

```python
from pathlib import Path
import json

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.linalg import toeplitz, cholesky, solve_triangular
from scipy.optimize import minimize
from scipy.integrate import quad
from scipy.signal.windows import dpss
from statsmodels.tsa.arima_process import arma_acovf
from problem2 import periodogram, multitaper_spectrum

ROOT = Path(__file__).resolve().parents[1]
DATA_DIR = ROOT / "data"
FIGURE_DIR = ROOT / "png"
SEED = 70103
NAO_DATA_PATH = DATA_DIR / "nao.csv"
N = 1024


def load_nao(path=NAO_DATA_PATH, n=N):
    table = pd.read_csv(path).iloc[:n]
    columns = [column for column in table if column not in {"year", "month", "day"}]
    if len(columns) != 1 or len(table) != n:
        raise ValueError("Expected n rows and one index column")
    values = pd.to_numeric(table[columns[0]], errors="raise").to_numpy()
    if not np.isfinite(values).all():
        raise ValueError("Missing values in the selected segment")
    dates = pd.to_datetime(table[["year", "month", "day"]])
    if not (dates.diff().dropna() == pd.Timedelta(days=1)).all():
        raise ValueError("The selected data are not sampled daily without gaps")
    return values


def spectral_density(frequencies, parameters):
    amplitude, decay, floor = parameters
    return amplitude * np.exp(-decay * np.abs(frequencies)) + floor


def whittle_likelihood(dft, parameters):
    dft = np.asarray(dft)
    n = len(dft)
    frequencies = np.fft.fftfreq(n)
    if np.any(np.asarray(parameters) <= 0):
        return np.inf
    selected = np.arange(1, (n + 1) // 2)
    model = spectral_density(frequencies[selected], parameters)
    objective = np.sum(np.log(model) + np.abs(dft[selected]) ** 2 / model)
    if n % 2 == 0:
        nyquist = spectral_density(np.array([0.5]), parameters)[0]
        objective += 0.5 * (np.log(nyquist) + np.abs(dft[n // 2]) ** 2 / nyquist)
```

```
    return float(objective)


def fit_spectrum(dft):
    n = len(dft)
    variance = float(np.sum(np.abs(dft) ** 2) / n)
    fits = []
    bounds = [(-16, 16), (np.log(0.1), np.log(1000)), (-20, 10)]
    for decay in (10.0, 25.0, 60.0, 120.0):
        initial = np.log([max(variance * decay / 2, 1e-6), decay, max(variance * 0.001, 1e-8)])
        fit = minimize(lambda z: whittle_likelihood(dft, np.exp(z)), initial,
                       method="L-BFGS-B", bounds=bounds, options={"maxiter": 1000, "ftol": 1e-12})
        if fit.success and np.isfinite(fit.fun):
            fits.append(fit)
    if not fits:
        raise RuntimeError("Whittle optimization did not converge")
    result = min(fits, key=lambda fit: fit.fun)
    return np.exp(result.x), result


def spectrum_to_acf(parameters, n=N):
    amplitude, decay, floor = parameters
    h = np.arange(n)
    acf = 2 * amplitude * decay * (1 - (-1.0) ** h * np.exp(-decay / 2)) / (decay ** 2 + (2 * np.pi * h)
     ** 2)
    acf[0] += floor
    return acf


def gaussian_log_likelihood(series, acf):
    series = np.asarray(series, dtype=float)
    factor = cholesky(toeplitz(acf[:len(series)]), lower=True)
    standardized = solve_triangular(factor, series, lower=True)
    return float(-0.5 * (len(series) * np.log(2 * np.pi) + 2 * np.log(np.diag(factor)).sum() +
     standardized @ standardized))


def main():
    FIGURE_DIR.mkdir(parents=True, exist_ok=True)
    values = load_nao()
    mean = float(values.mean())
    series = values - mean
    dft = np.fft.fft(series) / np.sqrt(N)
    parameters, fit = fit_spectrum(dft)
    acf = spectrum_to_acf(parameters)
    f, raw = periodogram(series)
    _, mt = multitaper_spectrum(series, dpss(N, 4, Kmax=7, norm=2))
    model = spectral_density(f, parameters)
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
    axes[0].plot(np.arange(N), values, linewidth=0.8)
    axes[0].set_xlabel("Day from 1979-01-01")
    axes[0].set_ylabel("Supplied index")
    axes[1].semilogy(f[1:], raw[1:], alpha=0.35, linewidth=0.7, label="Periodogram")
    axes[1].semilogy(f[1:], mt[1:], label="DPSS NW=4, K=7")
```

```
    axes[1].semilogy(f[1:], model[1:], "k--", label="Whittle fit")
    axes[1].set_xlabel("Frequency (cycles/day)")
    axes[1].set_ylabel("Two-sided spectral density")
    axes[1].legend(fontsize=8)
    fig.tight_layout()
    fig.savefig(FIGURE_DIR / "problem3_whittle_fit.png", dpi=170, bbox_inches="tight")
    plt.close(fig)
    empirical = np.array([series[:N - h] @ series[h:] / N for h in range(81)])
    fig, ax = plt.subplots(figsize=(8, 4))
    ax.plot(np.arange(81), empirical / empirical[0], label="Empirical")
    ax.plot(np.arange(81), acf[:81] / acf[0], label="Spectral model")
    ax.set_xlabel("Lag (days)")
    ax.set_ylabel("Autocorrelation")
    ax.legend()
    fig.tight_layout()
    fig.savefig(FIGURE_DIR / "problem3_acf.png", dpi=170, bbox_inches="tight")
    plt.close(fig)
    seasonal_lag = 365
    differences = series[seasonal_lag:] - series[:-seasonal_lag]
    h = np.arange(len(differences))
    extended = spectrum_to_acf(parameters, N + seasonal_lag)
    difference_acf = 2 * extended[h] - extended[h + seasonal_lag] - extended[np.abs(h - seasonal_lag)]
    old_ar = np.array([0.6642, 0.3518, -0.2324, 0.0837])
    old_ma = np.array([1.1556, 0.3247])
    old_acf = arma_acovf(np.r_[1, -old_ar], np.r_[1, old_ma], nobs=len(differences), sigma2=0.1956)
    exact_full = gaussian_log_likelihood(series, acf)
    exact_differences = gaussian_log_likelihood(differences, difference_acf)
    exact_old = gaussian_log_likelihood(differences, old_acf)
    quadrature = {}
    for lag in (0, 1, 10, 100, 1023):
        numeric = 2 * quad(lambda frequency: spectral_density(frequency, parameters) * np.cos(2 * np.pi *
 lag * frequency),
                           0, 0.5, epsabs=1e-9, limit=1000)[0]
        quadrature[str(lag)] = {"analytic": float(acf[lag]), "numeric": float(numeric)}
        if not np.isclose(numeric, acf[lag], atol=1e-7, rtol=1e-6):
            raise AssertionError("Spectral inversion check failed")
    results = {"n": N, "column": pd.read_csv(NAO_DATA_PATH, nrows=1).columns[-1],
               "sample_mean": mean, "parameters": parameters.tolist(), "whittle_objective": float(fit.fun)
,
               "optimizer_success": bool(fit.success), "variance_model": float(acf[0]),
               "variance_empirical": float(np.mean(series ** 2)),
               "exact_raw_log_likelihood": exact_full,
               "common_difference_n": len(differences),
               "stationary_difference_log_likelihood": exact_differences,
               "old_sarima_difference_log_likelihood": exact_old,
               "spectral_inversion": quadrature}
    (ROOT / "problem3_results.json").write_text(json.dumps(results, indent=2))
    print(json.dumps(results, indent=2))


if __name__ == "__main__":
    main()
```